Optimizing DNNs Model Partitioning for Enhanced Performance on Edge Devices

Md Al Maruf[†,*], Akramul Azim[†]
[†] Ontario Tech University

Abstract

Deep Neural Networks (DNNs) have proven effective in various applications due to their dominant performance. However, integrating DNNs into edge devices remains challenging due to the large size of the DNN model, which requires efficient model parallelization and workload partitioning. Previous attempts to address these challenges have focused on data and model parallelism but have fallen short in terms of finding the optimal DNN model partitions for efficient distribution, considering available resources.

This paper presents a pipelined DNN model parallelism framework that improves the performance of DNNs on edge devices. The framework optimizes DNN model training by determining the optimal number of partitions based on available edge resources. This is achieved through a combination of data and model parallelism techniques, which efficiently distribute the workload across multiple processors to reduce training time. The framework also includes a task controller to manage computing resources effectively. The experimental results demonstrate the effectiveness of the proposed approach, showing a significant reduction in the model training time compared to a baseline model AlexNet.

Keywords: Machine Learning, DNN Model Partitioning, Pipelined Model Parallelism, Edge Computing

1. Introduction

The integration of DNNs in Cyber-physical Systems (CPS) [1] such as autonomous driving [2], robotics, unmanned aerial vehicles (UAVs), industrial automation, and the internet of things (IoT) has been extensively increasing due to their ability to perform complex tasks accurately. With this increasing demand for intelligent systems, the architecture of computing devices is also evolving to meet application requirements like faster computation for real-time decision-making [3, 4]. Executing DNNs on remote servers (e.g., cloud) can introduce substantial latency due to the need to transmit large data over networks, leading to slower processing times. For example, in harsh environments where UAVs operate, communication and processing with remote servers can be impacted by weather [5]. Edge computing responds to these demands, enabling the distribution of computational capabilities to edge devices located near the data source, thereby reducing latency and enhancing the response time of applications.

Deploying DNN models on edge devices (e.g., embedded systems) presents various challenges, including the limited computational power and memory of edge devices, which can often prevent the deployment of large DNN models entirely. For example, Convolution Neural Networks (CNNs), another type of DNN, can be large and computationally intensive, which makes it challenging to deploy the entire CNN model on a single core edge device [6]. Traditionally, machine learning-based CPS applications were run sequentially on a single-core processor or device [7]. However, the increasing demand for faster computation in advanced applications necessitates parallel computing. Current approaches adopt data and model parallelism to train compute-intensive Machine Learning (ML) models on large-scale data sets, thus minimizing the overall training time and communication latency. Despite the availability of various parallelism techniques, the adoption of parallel computing for large DNN models on edge devices still remains challenging. To take advantage of the multiprocessor edge devices for DNN deployment, the applications must possess a sufficient degree of parallelism, enabling their computation to be distributed across different processors or devices.

[*] md.maruf@ontariotechu.net

To address this challenge, researchers have proposed various approaches for DNN model partitioning on edge devices. The objective of model partitioning is to divide the DNN model into smaller sub-modules that can run in parallel on multiple edge devices, with the goal of improving performance and reducing execution time. Several recent studies [8–12] have explored the possibility of DNN partitioning on different computing devices (such as edge devices and CPUs/GPUs) to improve training and inference times. Various partitioning algorithms, including estimation-based, structural adaptation-based, and measurement-based, have been proposed to run DNN models on different devices [11]. The main goal of these techniques is to cut the DNN model at a single point and offload the more computationally intensive part to another device for acceleration. For example, Chuang Hu et al. [12] proposed a min-cut-based algorithm to partition and offload DNNs in both edge and cloud environments. Similarly, cloud and edge-assisted approaches [6, 13] divide the DNNs into two parts to offer local and remote computation. However, previous studies have not addressed the ideal number of partitions for the DNN model concerning layer dependencies, device computing power, and communication latency.

In this study, we aim to find the optimal partitions of a DNN model across multiple edge devices. This task is particularly challenging due to the interdependence of DNN layers and the resource limitation of edge devices. Therefore, an efficient partitioning algorithm is essential to identify the best partition of a DNN model based on the available edge devices, as it helps to optimize resource utilization [14] and improve the overall performance of the model on edge devices. While DNN layers can be split into multiple processors, executing them without considering the layer's architectural dependency and communication latency may result in degraded performance. Additionally, model parallelism may lead to poor resource utilization if the execution schedule is not configured correctly. Thus, it is necessary to determine the optimal number of partitions and implement an effective task execution strategy to minimize model training time and avoid resource wastage. A pipeline-based task execution approach is particularly beneficial for model training since it enables orderly forward and backward passes, improving resource usage.

This paper presents a pipelined DNN model parallelism framework with an optimal DNN partitioning algorithm for enhancing application performance on edge devices. The proposed approach seeks to improve the performance of DNNs training on edge devices by considering the available resources, such as processing power. It divides the DNN model into smaller sub-modules, which can be executed in parallel on multiple edge devices, and leverages pipelining to improve performance further. Our study shows that combining data and pipelined model parallelism yields enhanced performance in model partitioning. In particular, pipeline execution of sub-modules significantly accelerates the training process for layers that have input dependencies. The main contribution of this work is determining the optimal number of partitions of DNN models that can be distributed among edge devices based on resource availability. Our comprehensive experiments demonstrate that the proposed model partitioning approach can significantly improve the DNN training time on edge devices compared to sequential execution and other state-of-the-art approaches.

The paper is structured as follows: Section 2 covers background and related works on model parallelism. Section 3 presents the problem statement, while Section 4 describes the system model and assumptions. Section 5 presents the proposed framework, and Section 6 discusses the experimental results. Section 7 addresses potential threats to validity, and Section 8 concludes the paper.

2. Background and Related Works

In distributed parallel computing, architectural dependencies, and communication latency are critical factors that affect performance [15]. Neglecting these aspects may result in unbalanced workload distribution and inefficient resource utilization. Model parallelism can improve performance but also presents challenges in partitioning and coordinating DNN models across multiple edge devices. Limited bandwidth and computational capacity of

edge devices may further reduce parallelism efficiency. Motivated by these challenges, this paper focuses on optimizing DNN model partitioning and pipelined model parallelism to improve application performance on edge devices.

FLOPs: FLOPs (floating-point operations) is a measure of the number of arithmetic operations (such as additions, subtractions, multiplications, and divisions) performed by a neural network during training or inference. It is used to estimate the computational cost of a neural network and is often used as a metric for comparing the efficiency of different models. For example, in the case of AlexNet [16] deep convolutional neural network, we can estimate the FLOPs required to compute the output of each layer by counting the number of arithmetic operations performed by the layer. The first convolutional layer of AlexNet has 96 filters, each with a size of 11×11. The input to the layer is a $227 \times 227 \times 3$ image. The layer uses a stride of 4 and padding of size 0. To compute the output of the layer, we need to perform the following operations:

Calculation of FLOPs: We have to apply convolution operation to the input image using all filters. For example, Convolution layer one has 96 filters. For each filter, we perform 11×11 multiplications. Since the output size is $55 \times 55 \times 96$, the total number of operations:

$$\begin{aligned} \textit{FLOPs for Conv} &= \textit{Number of Kernel} \times \textit{Kernel Shape} \times \textit{Output Shape} \\ &= 96 \times (11 \times 11 \times 3) \times (55 \times 55) \approx 105\text{M } [conv1] \end{aligned}$$

$$\begin{aligned} \textit{FLOPs for Fully Connected Layer} &= \textit{Input Shape} \times \textit{Output Size} \\ &= (6 \times 6 \times 256) \times 4096 \approx 37\text{M } [FC6] \end{aligned}$$

- Number of Kernel: the number of filters in the first layer (96)
- Kernel Shape: the size of each filter (11 x 11 x 3)
- Output Shape: the size of the output tensor (55 x 55 x 96)

The total FLOPs for all AlexNet layers is approximately 725M [17], as shown in Table 1.

Layer	Input Size	Filter	Output Size	FLOPs
Conv1	227x227x3	11x11x3	55x55x96	105,705,600
Conv2	27x27x96	5x5x48	27x27x256	223,948,800
Conv3	13x13x256	3x3x256	13x13x384	149,520,384
Conv4	13x13x384	3x3x192	13x13x384	112,140,288
Conv5	13x13x384	3x3x192	13x13x256	74,760,192
FC6	6x6x256		4096	37,748,736
FC7	4096		4096	16,777,216
FC8	4096		1000	4,096,000
Total	-	-	-	**725,147,008**

Table 1. FLOPs calculation for AlexNet

Estimating Training time from FLOPs:

- *Calculate the total number of iterations:* Assuming a batch size of 128 and a total of 1000 images, we can estimate the number of iterations required to complete one epoch of training. Specifically, we will have $1000/128 = 7.8125$ iterations per epoch. Since we plan to train our model for 100 epochs, the total number of iterations required will be $7.8125 * 100 = 781.25$.
- *Calculate the total number of FLOPs:* As we calculated before, the total number of FLOPs required to process one image through the AlexNet model is approximately 725 million. Since we are using a batch size of 128, the total number of FLOPs per iteration is 725 million$*$ 128 = 92.8 billion. Therefore, the total number of FLOPs required for training is 92.8 billion $*$ 781.25 = 72.4 trillion FLOPs.
- *Estimate the training time:* Given a Raspberry Pi 4 with 4GB RAM and 1.5 GHz quad-core ARM Cortex-A72 CPU, the peak performance is around 2.8 GFLOPS.

Each core can perform two floating-point operations per cycle, leading to a theoretical 3.0 GFLOPs; however, considering real-world factors, we assume a 2.8 GFLOPs peak performance. We can estimate the training time by dividing the total number of FLOPs by the processing power of the device:

$$\begin{aligned} \textit{Training time} &= \textit{Total FLOPs/Device Performance} \\ &= 72.4 \text{ trillion}/2.8 \text{ billion FLOPS} \approx 25{,}857 \text{ seconds} \end{aligned}$$

2.1. **Related Works**

In recent years, several pipeline-based model parallelism techniques have been proposed to accelerate the training of Deep Neural Network (DNN) models, including PipeDream-2BW [18], PipeDream [19], and GPipe [20]. These methods aim to split large DNN models across multiple machines for efficient computation.

PipeDream partitions the DNN layers into different stages and distributes them across interconnected computing devices. It employs an asynchronous pipeline mechanism, which requires significant memory to store intermediate model parameters. The forward passes are executed and distributed asynchronously, followed by the backward passes. In contrast, GPipe splits the minibatch of input data into multiple micro-batches and employs a pipeline strategy to execute these micro-batches across various devices. It updates gradients synchronously during backward passes and holds a single-weight version while flushing some results to free memory. However, this approach may lead to increased computation overhead due to the need to recompute intermediate results for backward propagation. PipeDream-2BW proposes a double-weight update mechanism to reduce the memory footprint, but it still experiences parameter staleness issues [21] and computation convergence that depends on the GPU platform.

Recent studies on pipelined model parallelism, such as those presented in [20, 22, 23], highlight the need for model parallelism in handling large DNNs that may not fit on a single machine's memory. However, implementing parallelism across different devices poses several performance issues, including complexity. For instance, PipeDream prioritizes maximum throughput, leading to high memory demand, which is impractical for embedded systems. Nonetheless, researchers have explored various memory-efficient model parallelism techniques that maintain model performance. PipeMare and PipeDream-2BW [18] are examples of such techniques that are more memory-efficient than PipeDream. Furthermore, DAPPLE [23] shows superior experimental results in terms of memory, speed up, and convergence compared to GPipe [20]. While these studies have demonstrated improved results in terms of time-to-accuracy, this paper's primary focus is to explore the partitioning of large DNN models to enhance their performance, specifically in terms of model training time on edge devices.

3. **Problem Statement**

The optimization problem for partitioning a DNN model M across a set of N edge devices with varying computational capacity, such that the total execution time of the model is minimized, can be mathematically formulated as:

$$\min_{P} \sum_{i=1}^{L} \sum_{k=1}^{N} x_{ik} \cdot T_{ik} \tag{3.1}$$

Where $P = \{x_{ik}\}$ is a set of binary variables that indicate the assignment of each layer i to each edge device k, subject to the following constraints:

Capacity constraint: The computational workload of each layer assigned to an edge device must not exceed the computational capacity of that device:

$$\sum_{i=1}^{L} F_i \cdot x_{ik} \leq C_k, \quad \forall k \in 1, 2, \ldots, N \tag{3.2}$$

Communication constraint: The communication time for each layer assigned to an edge device must not exceed the worst-case execution time of the layer on an edge device with minimum computational capacity C_{worst}:

$$\sum_{i=1}^{L} \Delta_{ik} \cdot x_{ik} \leq T_{worst}, \quad \forall i, k \tag{3.3}$$

Partitioning constraint: Each layer must be assigned to exactly one edge device:

$$\sum_{k=1}^{N} x_{ik} = 1, \quad \forall i \in 1, 2, \ldots, L \tag{3.4}$$

Here, F_i is the number of FLOPs of layer i in the DNN model M, Δ_{ij} is the communication overhead between layer i and edge device k, which includes both the communication costs associated with partitioning the DNN layers across edge devices and the communication costs incurred during the training process, such as exchanging weight updates and aggregating them among workers. Moreover, C_k is the computational capacity of the k-th edge device, D is the input data size, T_{ik} is the execution time of layer i on edge device k, T_{worst} is the worst-case execution time of layer i on an edge device with minimum computational capacity C_{worst} in the considered network, and L is the total number of layers in the DNN model. The objective function seeks to minimize the total execution time of the model across all edge devices by finding the optimal assignment of layers to edge devices, subject to capacity and communication constraints. The partitioning constraints ensure that each layer is assigned to exactly one edge device. Despite the potentially NP-hard nature of the optimization problem, it can be solved using mixed-integer linear programming (MILP) solvers such as IBM CPLEX or Gurobi, which efficiently handle binary variables, linear constraints, and integer constraints. We can apply problem-specific heuristics to further improve the solution process. The obtained solution can be used to optimize the performance of DNN models on edge devices with limited computational resources and bandwidth.

4. System Model and Assumptions

The system model for DNN model partitioning in pipelined model parallelism is based on a heterogeneous multiprocessor edge device platform designed for DNN deployment. The primary objective is to optimally partition the DNN model into sub-models and distribute them among available edge devices or multi-core processors based on their computational capacities to achieve parallelism. We assume that a multiprocessor edge device is equipped with multiple cores to facilitate parallel computing. It can be a general-purpose or embedded processor.

To implement this approach, the AlexNet CNN architecture is used as a baseline. The proposed approach involves partitioning the CNN model into sub-modules and mapping them to parallel implementation using the task graph model. The parallel execution of these sub-modules should generate the same output as running the program on a single processor. The design architecture for the proposed DNN model partitioning for cyber-physical or embedded applications is illustrated in Figure 1. It is assumed that DNN applications require faster model training to meet their task requirements, which is facilitated by N number of edge devices or processors. For executing partitioned sub-modules, a task graph, represented as a directed acyclic graph (DAG), captures computational tasks, communication costs, and dependencies between the DNN model layers. The proposed DNN model partitioning algorithm determines the optimal number of partitions for the DNN model while considering the available edge resources and their communication times. The tasks controller then

analyzes the partitioned modules and schedules their execution in a pipeline fashion to reduce computation time.

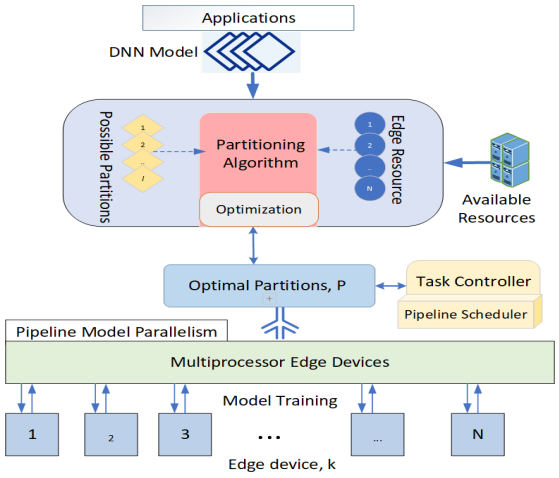

Figure 1. Design overview for DNN model execution on edge devices

5. **Proposed Approach and Methodology**

This paper proposes a framework that finds the number of partitions for parallelizing the Deep CNN model on edge devices. The partitioned modules are distributed among the available edge devices based on the computing capacity of the processors and the inter-process communication latency. Our approach combines data parallelism and model parallelism to achieve faster computation than traditional sequential DNN model training. In the following sections, we present the details of our proposed algorithm and methodology for pipelined model parallelism.

5.1. **Model Partitioning**

The fundamental aspect of CNN model parallelism is its layers, illustrated in Figure 2. These layers contain convolution and fully connected components that can be distributed among multiple processors to improve model training efficiency. However, dividing all the layers equally may result in increased communication overhead during training. Therefore, identifying the ideal number of partitions is a crucial aspect of deep learning model parallelism. We propose Algorithm 1 to determine the optimal partitions for running CNN layers on edge devices. The proposed algorithm aims to optimize the training time of a deep neural network (DNN) model by partitioning it across a set of edge devices. The algorithm requires the DNN model, the number of edge devices, and the computational capacity and communication overhead of each device as input. Initially, the algorithm computes the FLOPs of each layer and the worst-case execution time of the model on a single-edge device with the minimum capacity. It then initializes the optimal partition and best objective value to empty and infinity, respectively.

Next, the algorithm sets up a mixed-integer linear programming (MILP) problem for each layer and edge device with binary variables. The objective function is set to minimize the total execution time of the model, taking into account the FLOPs of each layer and the time taken to execute the layer on each edge device. Capacity and communication constraints are added to prevent the computational workload and communication time from exceeding the available resources.

Algorithm 1 Finding Optimal DNN Model Partitions for Edge Devices

Require: DNN model M, number of partitions S, set of edge devices N, computational capacity C_k, Input data size D, communication overhead Δ_{ik},
Ensure: Optimal partitions of Model M for edge devices

```
Compute FLOPs F_i of each layer in M
Compute total FLOPs F_total = sum_{i=1}^{L} F_i
Compute the execution time T_i for each layer i based on the FLOPs and the computational capacity of the edge devices
Compute the worst-case execution time T_worst of M on a single edge device with capacity C_worst = min_k C_k
Initialize optimal partitions P* = {} and best objective value obj* = ∞
for n = 1 to S do
    Initialize binary variable x_{i,k} for each layer i and edge device k
    Define objective function obj_n = sum_{i=1}^{L} sum_{k∈N} T_i · x_{i,k}
    Define capacity constraint sum_{i=1}^{L} F_i x_{i,k} ≤ C_k for all k ∈ N
    Define communication time T_{comm,ik} = Δ_{ik} D / C_worst for all i and e
    for i = 1 to L do
        for k = {1, 2} ∈ N do
            Define communication constraint T_{comm,i,k_1} x_{i,k_1} ≥ T_{comm,i,k_2} x_{i,k_2}
    Solve MILP problem to find optimal partition P_n and objective value obj_n
    if obj_n < obj* then
        Set P* = P_n and obj* = obj_n
return P*
```

The algorithm then solves the MILP problem for each partition to determine the optimal partition and corresponding objective value. If the objective value is lower than the current best objective value, the optimal partition and best objective value are updated accordingly. The algorithm continues this process for each partition and returns the optimal partition with the minimum execution time. This approach can enhance the performance of DNN models on edge devices with limited computational resources and bandwidth.

5.1.1. **An illustrative Example**

To illustrate our proposed partitioning algorithm, we provide an example of how to optimally partition the AlexNet DNN model across four edge devices with computational capacities of approximately 200M, 100M, 70M, and 50M FLOPs/s. We first calculate the FLOPs for each layer of the AlexNet model, as shown in Table 1, with a total of approximately 725M FLOPs for all layers. We then compute the execution time for each layer on each device using the FLOPs and the computational capacity of each device, as presented in Table 2. To determine the optimal partitions for the AlexNet DNN model, we continue

Layer	**FLOPs**	**Device 1**	**Device 2**	**Device 3**	**Device 4**
1	238,878,720	1.19 s	2.37 s	3.56 s	4.74 s
2	1,074,042,624	5.37 s	10.73 s	16.10 s	21.46 s
3	231,211,264	1.16 s	2.32 s	3.47 s	4.63 s
4	231,211,264	1.16 s	2.32 s	3.47 s	4.63 s
5	173,408,256	0.87 s	1.73 s	2.60 s	3.46 s
6	86,704,640	0.43 s	0.87 s	1.30 s	1.73 s
7	41,943,040	0.21 s	0.42 s	0.63 s	0.84 s
8	2,097,152	0.01 s	0.02 s	0.03 s	0.04 s

Table 2. Execution time of AlexNet layers on different edge devices

with the following steps:

- We compute the worst-case execution time of AlexNet on a single edge device with a capacity of $C_{worst} = \min(C_k)$: $C_{worst} = 50,000,000$ FLOPs/s (capacity of Device 4), and $T_{worst} = F_{total}/C_{worst} = 14.503$ s.
- For each partition n (in this case, we try with $S = 4$ possible partitions), we define the capacity and communication time (assuming $0.1s$) constraints.
- Finally, we run the MILP optimization, which yields the optimal partition P^* and the best objective value obj^*. However, many factors, such as the choice of objective function and communication constraints, can affect the performance of the partitioning algorithm in practice.

Partition Number	**Layers**	**Devices**	**Execution Time**
1	1, 2, 3, 4, 5	1	12.03 s
2	6, 7	2	0.137 s
3	8	3	0.007 s
		Total	**12.202 s**

Table 3. Optimal partitioning of AlexNet DNN model over four distinct edge devices

The optimal partitioning result of the AlexNet model is presented in Table 3. The optimization algorithm has determined that the optimal partitioning of the AlexNet model is to divide it into three partitions, with the first partition consisting of layers 1 to 5, the second partition consisting of layers 6 and 7, and the third partition consisting of layer 8. This partitioning results in an overall execution time of 12.202 seconds, which is faster than the worst-case execution time on a single-edge device. This shows the advantage of using edge computing and optimal partitioning to speed up the execution of DNN models.

5.2. **Pipelined Execution**

According to the designed architecture shown in Figure 1, the task controller is responsible for distributing the partitioned modules of the CNN model to different processors based on edge resources. Algorithm 1 is responsible for finding the optimal partitions of the model, as shown in Figure 2 for the AlexNet model. This model consists of five consecutive convolutional layers and three fully connected layers. For better performance, data parallelism is more effective in training convolutional layers, while model parallelism is more suitable for dense or fully connected layers. The framework implements data parallelism by duplicating the convolution layers on computing processors to run input batch data in parallel. On the other hand, the fully connected layers are split into two parts, and model parallelism is applied to train them across the model dimension.

To achieve efficient data parallelism and optimize processor utilization when training the convolutional layers, we propose implementing a pipeline execution of mini-batch input data. Figure 2 illustrates this approach, where each processing core handles an input batch data and runs the partitioned model. For instance, if a multi-core edge device is assigned to execute a partitioned module containing all convolution layers, the first core sequentially runs the convolution layers using the first input batch data, while other processing cores begin computing subsequent input batches using the pipeline approach. As soon as the first core completes the execution of the last layer, the next available core immediately starts executing the next input batch.

5.3. **Model Training on Edge Devices**

The proposed approach in Figure 2 for DNN model training employs a dispatcher that sends input data batches to dense layers once convolution layers complete their training. Fully connected layer partitions receive the batch data for training the sub-module, and the

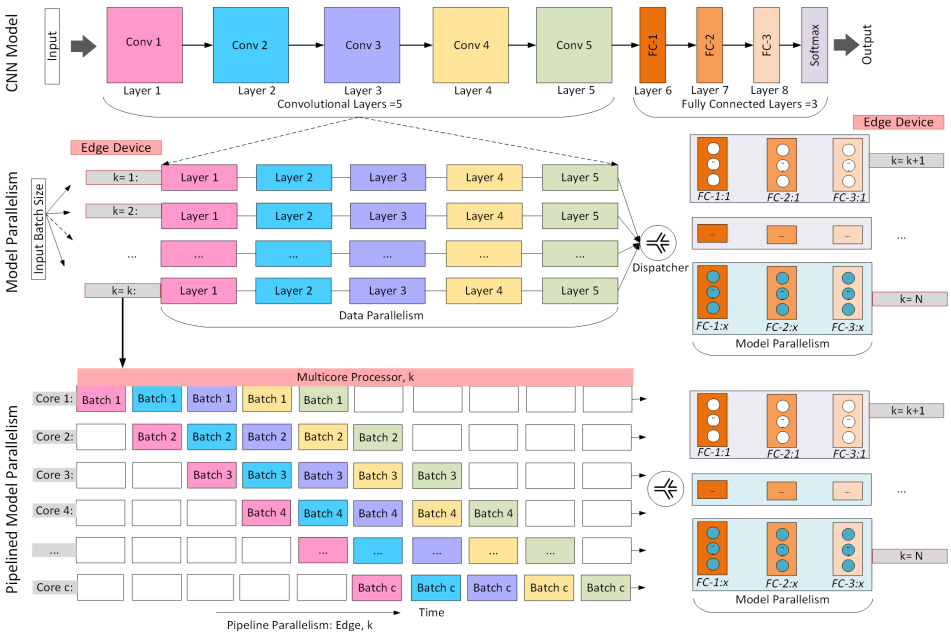

Figure 2. Pipelined model parallelism on edge devices

dispatcher sends the input batch data sequentially to maintain the forward and backward propagation. During training, the number of workers, w, is determined based on the number of available processors. Each worker is assigned to an edge device with the corresponding partitioned portion of the model. Workers update weights using gradient descent, and the primary worker aggregates weights from all other workers. The performance of the optimal number of partitions is evaluated based on the communication latency and execution time of each layer, and memory usage is monitored for under-utilization or overloading.

6. Experimental Results and Analysis

Environment Setup and Dataset: The proposed framework's performance was evaluated for image object detection on an embedded system functioning as an edge device. The experiments were conducted on various architectures, including AlexNet [16], ResNet [24], and VGG-16 [25], considering their relevance, computational complexity and popularity. Experiments were performed on the NVIDIA Jetson Nano [26], which is equipped with Quad-core ARM Cortex-A57 processors. The operating system used for the experiments was Ubuntu 20.04 LTS, and the Jetson Nano had 4 GB of 64-bit LPDDR4 memory. In this experiment, an image dataset [27] for plant leaf disease detection is utilized, consisting of over 50,000 images, each classified into one of 38 disease classes. The dataset is divided into training and testing sets, with 80% for training and 20% for testing, after resizing the images to 227×227.

6.1. Efficiency Evaluation of Proposed Framework

The purpose of this experiment is to assess the effectiveness of different CNN architecture partitioning techniques for model parallelism, with the goal of reducing the model training time. The training is performed using the python multiprocessing library, Ray [28], on a Jetson Nano board with an SGD optimizer, for 20 epochs, where each epoch has 200 steps with a batch size of 32. The net execution time is measured for each epoch as the model is partitioned into varying numbers of cores. The experiment is conducted on a Jetson Nano

board with four available processors. The proposed algorithm, Algorithm 1, is utilized to determine the optimal number of partitions for the AlexNet, ResNet50, and VGG-16 architectures. The maximum number of partitions is limited to four, due to the limited number of CPU cores, with the optimal number of partitions being found to be three, four, and four, respectively. The net execution times for the three architectures are analyzed for different numbers of partitions.

Networks	**Layers** (L)	**FLOPs**	**Partitions** (S)	**Epochs**	**Pipelined Parallel Training Time**	**Sequential Training Time**	**Accuracy (Pipelined)**
AlexNet [16]	8	725M	3	20	3129.4s	6950.7s	96.1%
ResNet50 [24]	50	3.8G	4	20	1574.5s	4956.5s	97.6%
VGG-16 [25]	16	16G	4	20	5525.6s	10792.3s	90.3%

Table 4. Pipelined model parallelism performance for different CNN networks

The optimized function given in Equation 3.1 was used to calculate the training time T_{ij} of each layer, as shown in Table 4. The results reveal that pipelined parallel computing for AlexNet, with three optimal splits, takes approximately 3129.4s, yielding a speed-up of 2.3 times over serial execution on a single core. Similarly, the ResNet50 and VGG-16 models show the lowest execution time when split into four CPU cores, with speed-ups of 3.2 times and 2.0 times, respectively, over sequential executions.

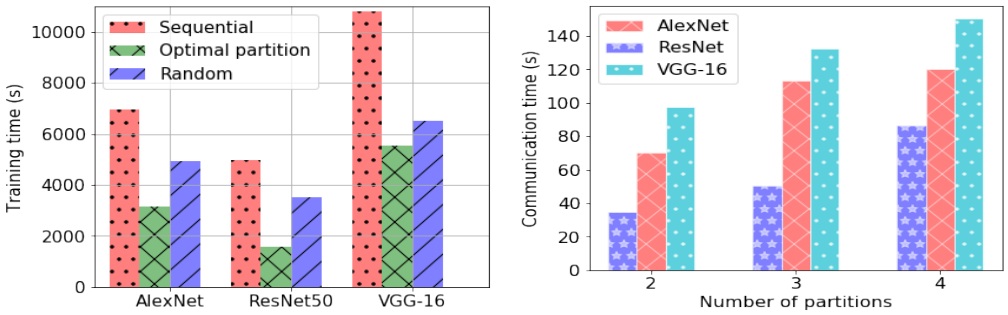

Figure 3. Comparison of (a) training time for partitioning model randomly (b) communication time for different number partitions

To evaluate the effectiveness of pipelined model parallelism for optimal partitioning, we compared the training time of the random partitioning approach, which partitions the model randomly. Figure 3(a) shows the comparison of training time among the non-partitioning (sequential), random, and optimal partitioning approaches. It is observed that the optimal partitioning approach minimizes the training times for all three CNN networks. Figure 3(b) illustrates the communication latency for different partitions over available edge devices or processors. The results indicate that the latency rises with the increasing number of model splits, particularly for large DNN models with a significant number of trainable parameters. VGG-16, with its high number of parameters, demonstrates the highest latency. The experiment demonstrates an average speed-up of 2.5 times with pipelined model parallelism over sequential executions. The potential of pipelined model parallelism to enhance the performance of DNN applications in multiprocessor edge devices is evident. We expect the acceleration to improve further with increasing training epochs for larger DNN models.

7. Threats to Validity

To ensure the scope and applicability of our work, we identify potential threats to the validity of our proposed approach.

Generalizability to all DNN applications: Our approach is focused on the CNN architecture, which is a specific type of DNN. Therefore, the effectiveness of our model-splitting approach may be limited to architectures that have a similar structure to CNN.

Applicability to GPU-enabled edge device: While GPU support is available, our work mainly focuses on supporting multiprocessor edge devices that predominantly use multi-core CPUs. This is to enable the reuse of existing embedded systems and avoid additional costs. However, our approach can be extended to support multi-GPU edge devices as well.

8. Conclusion

This paper introduces a pipeline-based DNN model parallelism framework that improves the performance of applications by utilizing an optimal DNN partitioning algorithm to distribute partitions across different edge devices. Our experimental results, based on the AlexNet architecture, demonstrate that the pipelined model parallelism approach significantly reduces the net execution time for optimal partitions. In the future, we aim to explore the feasibility and effectiveness of dynamic partitioning based on computational load.

9. Acknowledgements

This research was supported by a NSERC grant of Canada.

References

[1] X. Chen, A. Azim, X. Liu, S. Fischmeister, and J. Ma. "DTS: dynamic TDMA scheduling for networked control systems". In: *Journal of Systems Architecture* 60.2 (2014), pp. 194–205.

[2] X. Wang, X. Wu, S. Cheng, J. Shi, X. Ping, and W. Yue. "Design and experiment of control architecture and adaptive dual-loop controller for brake-by-wire system with an electric booster". In: *IEEE Transactions on Transportation Electrification* 6.3 (2020), pp. 1236–1252.

[3] K. Cao, S. Hu, Y. Shi, A. W. Colombo, S. Karnouskos, and X. Li. "A survey on edge and edge-cloud computing assisted cyber-physical systems". In: *IEEE Transactions on Industrial Informatics* 17.11 (2021), pp. 7806–7819.

[4] M. Al Maruf, A. Singh, A. Azim, and N. Auluck. "Faster Fog Computing Based Over-the-Air Vehicular Updates: A Transfer Learning Approach". In: *IEEE Transactions on Services Computing* 15.6 (2022), pp. 3245–3259.

[5] M. Jouhari, A. K. Al-Ali, E. Baccour, A. Mohamed, A. Erbad, M. Guizani, and M. Hamdi. "Distributed CNN inference on resource-constrained UAVs for surveillance systems: Design and optimization". In: *IEEE Internet of Things Journal* 9.2 (2021), pp. 1227–1242.

[6] J. Na, H. Zhang, J. Lian, and B. Zhang. "Partitioning DNNs for Optimizing Distributed Inference Performance on Cooperative Edge Devices: A Genetic Algorithm Approach". In: *Applied Sciences* 12.20 (2022), p. 10619.

[7] R. Hilbrich. "How to Safely Integrate Multiple Applications on Embedded Many-Core Systems by Applying the "Correctness by Construction" Principle". In: *Advances in Software Engineering* 2012 (2012).

[8] Y. Xiang and H. Kim. "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference". In: *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE. 2019, pp. 392–405.

[9] J. Kim, Y. Park, G. Kim, and S. J. Hwang. "SplitNet: Learning to semantically split deep networks for parameter reduction and model parallelization". In: *International Conference on Machine Learning*. PMLR. 2017, pp. 1866–1874.

[10] T. Mohammed, C. Joe-Wong, R. Babbar, and M. Di Francesco. "Distributed inference acceleration with adaptive DNN partitioning and offloading". In: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE. 2020, pp. 854–863.

[11] F. McNamee, S. Dustdar, P. Kilpatrick, W. Shi, I. Spence, and B. Varghese. "The Case for Adaptive Deep Neural Networks in Edge Computing". In: *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. IEEE. 2021, pp. 43–52.

[12] C. Hu, W. Bao, D. Wang, and F. Liu. "Dynamic adaptive DNN surgery for inference acceleration on the edge". In: *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE. 2019, pp. 1423–1431.

[13] H.-J. Jeong, H.-J. Lee, C. H. Shin, and S.-M. Moon. "IONN: Incremental offloading of neural network computations from mobile devices to edge servers". In: *Proceedings of the ACM symposium on cloud computing*. 2018, pp. 401–411.

[14] R. Mamata and A. Azim. "Work-in-Progress: A Resource-Aware Optimization Model for Real-Time Systems Analysis and Design". In: *2022 International Conference on Embedded Software (EMSOFT)*. IEEE. 2022, pp. 9–10.

[15] M. A. Maruf and A. Azim. "Requirements-preserving design automation for multiprocessor embedded system applications". In: *Journal of Ambient Intelligence and Humanized Computing* 12.1 (2021), pp. 821–833.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton. "ImageNet classification with deep convolutional neural networks". In: *Communications of the ACM* 60.6 (2017), pp. 84–90.

[17] M. Shahshahani, P. Goswami, and D. Bhatia. "Memory optimization techniques for fpga based cnn implementations". In: *2018 IEEE 13th Dallas Circuits and Systems Conference (DCAS)*. IEEE. 2018, pp. 1–6.

[18] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, and M. Zaharia. "Memory-efficient pipeline-parallel dnn training". In: *International Conference on Machine Learning*. PMLR. 2021, pp. 7937–7947.

[19] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia. "PipeDream: generalized pipeline parallelism for DNN training". In: *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 2019, pp. 1–15.

[20] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, et al. "Gpipe: Efficient training of giant neural networks using pipeline parallelism". In: *Advances in neural information processing systems* 32 (2019).

[21] M. Tanaka, K. Taura, T. Hanawa, and K. Torisawa. "Automatic graph partitioning for very large-scale deep learning". In: *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE. 2021, pp. 1004–1013.

[22] A. Harlap, D. Narayanan, A. Phanishayee, V. Seshadri, N. Devanur, G. Ganger, and P. Gibbons. "Pipedream: Fast and efficient pipeline parallel dnn training". In: *arXiv preprint arXiv:1806.03377* (2018).

[23] S. Fan, Y. Rong, C. Meng, Z. Cao, S. Wang, Z. Zheng, C. Wu, G. Long, J. Yang, L. Xia, et al. "DAPPLE: A pipelined data parallel approach for training large models". In: *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 2021, pp. 431–445.

[24] K. He, X. Zhang, S. Ren, and J. Sun. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[25] K. Simonyan and A. Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).

[26] A. Menshchikov, D. Shadrin, V. Prutyanov, D. Lopatkin, S. Sosnin, E. Tsykunov, E. Iakovlev, and A. Somov. "Real-time detection of hogweed: UAV platform empowered by deep learning". In: *IEEE Transactions on Computers* 70.8 (2021), pp. 1175–1188.

[27] I. Z. Mukti and D. Biswas. "Transfer learning based plant diseases detection using ResNet50". In: *2019 4th International conference on electrical information and communication technology (EICT)*. IEEE. 2019, pp. 1–6.

[28] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, et al. "Ray: A distributed framework for emerging {AI} applications". In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 2018, pp. 561–577.